



University of
Nottingham
UK | CHINA | MALAYSIA

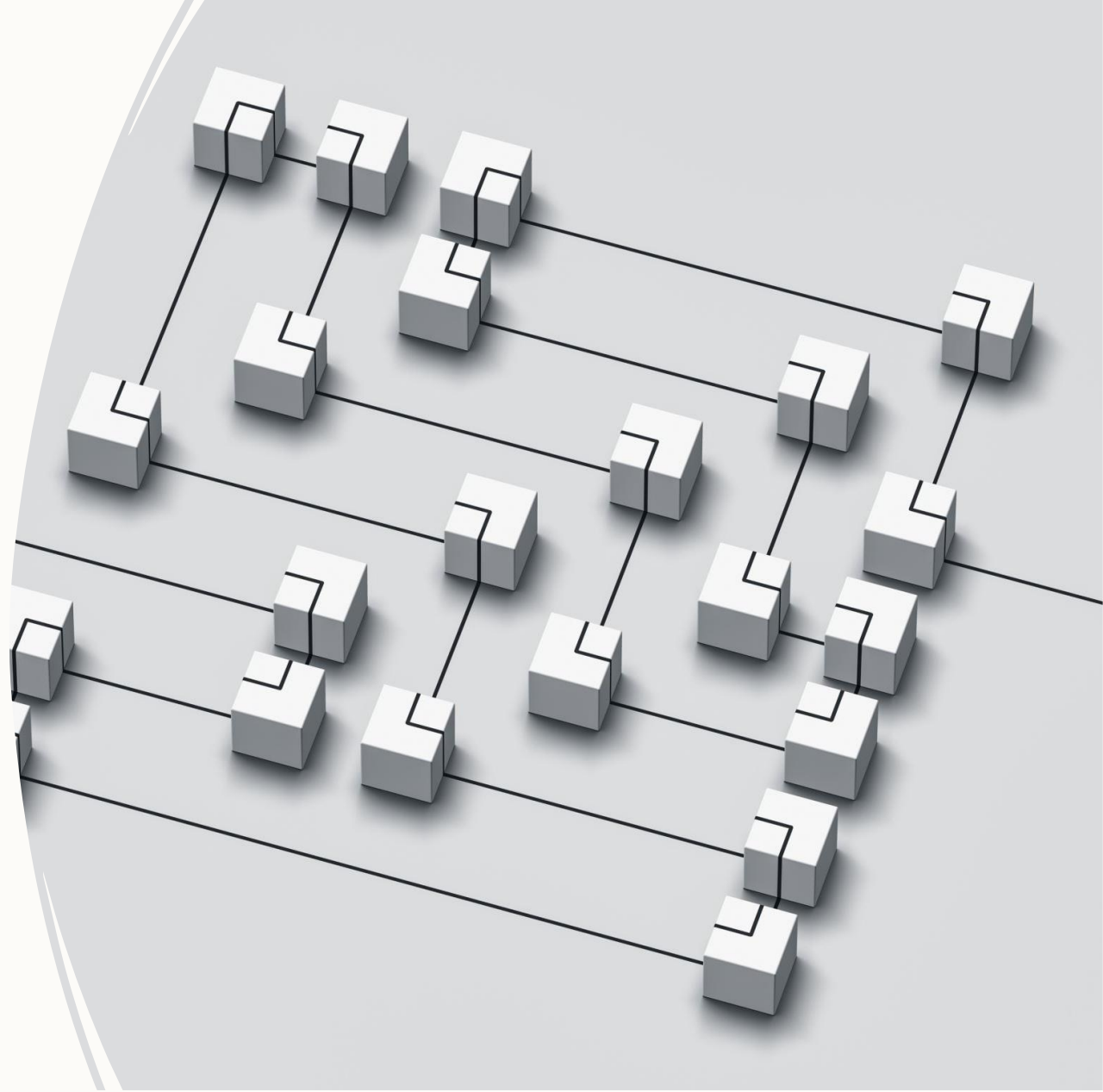
COMP 3056 Professional Ethics in Computing

Week 9 Trust, Safety and
Reliability



Learning outcomes

- Causes of Computer Failure
- Bugs and Public Safety
 - When computer failure can lead to harm or death
 - Accountability and Moral Responsibility
- Malware





University of
Nottingham

UK | CHINA | MALAYSIA

Trusting computer systems

Trust: Example One (Addition)

Scenario: You want the sum of 1,000 numbers. You get **three different answers** from **three different sources**:

- a) One person using **pencil and paper**
- b) One person using a **calculator**
- c) One person using dedicated 专业的电子表格软件 **spreadsheet software**

Qu: Which would you be most likely to trust?

Trust: Example Two (Translation)

Scenario: You need a Russian poem translated. You get **three different answers** from **three different sources**:

- a) You ask a friend, who is a **Russian major**, for a translation
- b) You use dedicated **translation software** to do it yourself
- c) You search the poem's title and **find a translation on the Internet**

Qu: Which would you be most likely to trust?

Trust

In some cases we are **right to trust** computers, in other cases we are **wrong**...

Today, we will:

探讨人们信任计算机的方式

- Explore ways in which people **trust** computers
- Examine how and why computers sometimes **fail** to **justify our trust** 研究计算机有时为何以及如何辜负我们的信任
考虑这种失败的后果
- Consider the **consequence** of such failure
讨论降低失败可能性的方法
- Discuss ways to **minimize failure** likelihood
讨论对失败的道德责任/问责制
- Discuss **moral Responsibility / accountability** for failures



University of
Nottingham

UK | CHINA | MALAYSIA

Computer Failure: Causes

Causes of Computer Failure

- **Hardware errors** 硬件故障包括：CPU 设计缺陷，内存位翻转（bit flip），硬盘坏道，传感器失灵，电源不稳定
一旦硬件输出错误信号，**软件再完美都没用**，因为数据源已经错了。
 - **Hardware malfunctions causing erroneous results** 硬件故障导致错误结果
- **Software errors** 软件错误可能包括：算法逻辑错误，条件判断错误，溢出（overflow/underflow），多线程竞争（race condition），错误处理不足，设计缺陷（architecture flaw）
 - **Software “bugs” in code or software design** 代码或软件设计中的“漏洞”
- **Computer solves the wrong problem** 这是非常重要的类型：软件本身逻辑完全正确但需求理解错了客户想做 A，程序员理解成 B 软件“完美无 bug”，但输出完全不是客户想要的
这不是代码错误，是**需求错误 / 需求误解**（requirements misunderstanding）
 - **Programmers incorrectly understand requirements of clients. Program does as intended (no bugs) but does not do what client wants and expects**
- **Misuse** 程序员对客户的需求理解有误。程序按预期运行（无漏洞），但并未完成客户想要和期望的任务
这是用户造成的错误，不是系统的错。
两种情况：**用错系统（wrong purpose）**例如：用 Excel 做高并发实时系统 崩溃
把正确系统用错（incorrect use）例如：输入错误格式、把单位搞错
误用/滥用
 - **Using system for the wrong purpose, or using system incorrectly for the right purpose. E.g., “Garbage in, garbage out”** 将系统用于错误的目的，或正确目的但使用方法不当。例如，“输入垃圾，输出垃圾”
- **Communication failure (in human interaction)** 沟通失败（人类交互中）
 - **User prompted for action misunderstands (e.g., airplane refuel, user assumes amount in gallons rather than litres – plane runs out of fuel!)** 用户被提示采取行动时产生误解（例如，给飞机加油，用户以为是加加仑而不是升——飞机没油了！
- **Malice** 恶意行为=人故意让系统失败
 - **Malicious software (virus, trojan, hacking ...) for profit, terrorism, warfare, “fun”**
恶意软件（病毒、木马、黑客攻击……）用于谋取利益、恐怖主义、战争、“寻开心”
恶意攻击包括：病毒（virus）蠕虫（worm）木马（trojan horse）间谍软件（spyware）勒索软件（ransomware）
黑客（hacking） 动机：盈利（如盗取信用卡）恐怖活动间谍 / 战争个人恶作剧或“炫技”

Case: Software Error

- ESA unmanned space flight: Rockets explode (\$3.5mn loss) 欧洲航天局无人太空飞行：火箭爆炸（损失 350 万美元）
可预防的错误：开发者赌错误不会发生---->典型的风险管理失败
- **Preventable**: developers gamble error won't occur
 - 代码复用Ariane4:浮点数转换为整数
 - **Reuse of Ariane 4 code**. Floating point (FP) numbers (used for large numbers) converted to integers
Ariane5加速度比Ariane4快得多，浮点数增长的更快
 - Ariane 5 accelerated much faster than Ariane 4, so FP values grow much quicker
64-bit浮点数转换为16-bit整数，数值不合适，造成算法溢出错误
 - Large 64-bit FP value converted to 16-bit integer.
Value couldn't fit, **causing arithmetic overflow error**
7个变量，只有4个变量通过错误处理进行了保护。其他变量在to-do list
 - Of 7 variables, only 4 protected by error-handling code. **Other variables on the “to-do” list!**

事故机制总结：

复用 Ariane 4 代码（没有验证是否适用于 Ariane 5）

Ariane 5 速度更快-->某飞行参数值急剧增大

程序强制把 64-bit FP 转成 16-bit int

数值太大-->溢出-->抛出硬件异常

异常未被捕获-->导航系统崩溃-->火箭自毁



Ariane 5 是软件错误 (software error) ，不是硬件错误

Case: Hardware Error

- Prof Nicely notified Intel about flaw in P5 chip P5芯片存在缺陷
浮点数除法运算偶尔会出现错误
 - Occasional flaw in division of one floating point (FP) by another
发生概率极低 (1:90 亿)，大多数用户难以察觉
 - Rare (1 : 9,000,000,000) and unlikely to be noticed by most users
但全球所有个人电脑的运算次数极其庞大！
 - But, number of calculations on all PCs in the world is huge!
- Intel already aware of bug, but argued: not important / affects few users 英特尔已知该缺陷，但辩称：无关紧要/影响用户极少
 - 缺陷被 CNN 报道，成为“新闻”
 - Bug was reported by CNN and became “news”
英特尔提出，若用户能证明需要高精度运算，可更换芯片
 - Intel offered to replace chips if users demonstrate a need for high precision
公众愤怒：英特尔有义务提供正常工作的产品；用户不应预测未来使用情况；用户应能信任计算机达到标准规定的精度
 - Public outrage: Intel had obligation to deliver working product; users should not have to predict future usage; users should be able to trust computers deliver precision defined by standards
- Result: Intel agreed to replace all chips 结果：英特尔同意更换所有芯片
 - Cost: \$500mn. Loss of public's trust and reputational damage 成本：5 亿美元。公众信任度下降，声誉受损



为什么这个案例重要？

这是

硬件错误 (hardware error)

的代表性案例

不是软件 bug，是 浮点除法单元 (hardware logic) 的问题。

说明“小概率错误”不能被忽视

尤其是全球使用规模大的基础硬件。

展现了“责任与公众信任 (trust)”的重要性

企业不重视问题 媒体曝光 声誉崩塌。

展示科技公司在遇到伦理问题时的典型失败回应

“证明你需要精度”是极其不负责任的说法。



Bugs and Public Safety

- When computer failure can lead to harm or death
- Accountability and Moral Responsibility

Safety-Critical Software (SCS) 安全关键型软件

若运行不正常会影响人员安全的软件

- Software that affect somebody's safety if fails to work properly

空中交通管制、核安全系统、医疗等

- Air traffic control, nuclear safety system, medical, etc.

创建、测试和维护 SCS 的标准比其他软件严格得多

- Standards of creating, testing, and maintaining SCS are much stricter than other software

全面测试（穷举测试）似乎是显而易见的答案，但软件中大量的决策点使这变得不可能-->不能做穷举测试，无法保证覆盖所有情况

- Exhaustive testing is the obvious answer, but large number of decision-points in software makes this impossible

软件还控制着机械和设备（SCS 不只是“显示结果”的软件，而是直接控制物理世界的设备。），因此

- Software also control machinery and equipment, therefore

实时软件系统（例如，在碰撞前避开物体）：实时性=错过时间=出事故

- Real-time software systems (e.g., avoid object before crash)

软件在多进程环境中执行（例如，加速、刹车、避让）：多进程+实时性=测试极其复杂

- Software executes in multi-process environments (e.g., accelerate, brake, avoid)

这些都极大地增加了测试的复杂性

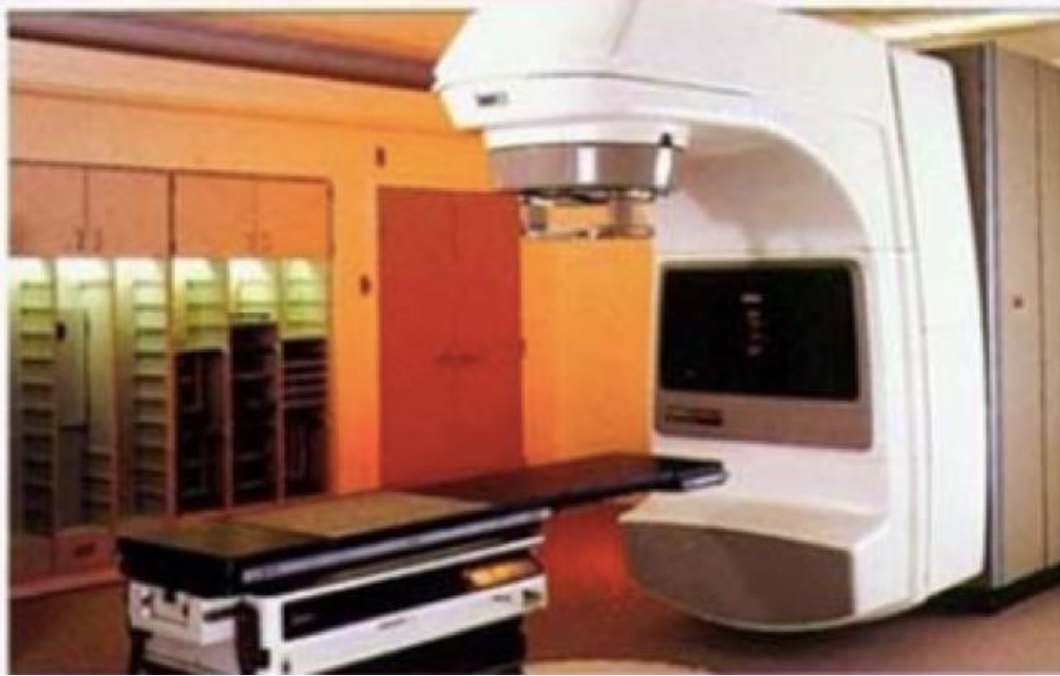
- These greatly complicate testing

软件错误 = 机器物理执行错误-->现实世界灾难

Towards Better Safety-critical Software

- Some improvements have been achieved
 - Better education and training of engineers, programmers, analysts 更好的工程师、程序员和分析师的教育与培训
更复杂的软件测试程序
 - More sophisticated software testing procedures
可能包括形式验证（如数学证明）
 - May include **formal verification** (like a mathematical proof)
 - Better maintenance 更好的维护
重新聚焦于预防而非检测/纠正
 - Refocus on prevention rather than detection/correction
模拟可能有好处（例如空难）
 - Simulation can be beneficial (e.g., air-disaster)
强调道德和专业教育
 - Emphasis on ethics and professional education

Therac-25 是一台用于癌症治疗的高功率医疗直线加速器。它大量依赖软件控制剂量，而移除了以前版本的硬件安全保护。由于软件错误、缺乏测试、界面设计缺陷以及对患者报告的不重视，在 1985 - 87 年间发生六起严重事故，导致至少三人因接受超过 1000 倍辐射剂量而死亡。该案例展示了安全关键软件失败的极端后果，成为软件工程和系统安全的经典反面教材。



CASE: THERAC-25 (1985-87)

- Machine designed for cancer treatments, known as a medical linear accelerator
- “25” refers to power, much higher than previous machines in series
- Heavy reliance on software for control (and safety)
- Six serious accidents, resulting in 3 deaths (>1,000x required dose!)
- Nancy Leveson, (1995) *Safeware: System Safety and Computers*, Addison-Wesley

Summary of Accidents

1985 年：一名女性因乳腺癌接受放射治疗。肩部严重灼伤

(1) 1985: Woman undergoing radiation therapy for breast cancer. Suffered severe burn to shoulder

- Patient reported severe pain 患者报告称疼痛难忍
医生不相信机器灼伤了她
- Physicians didn't "believe" machine burned her
其中一人建议涂抹外用乳膏
- One recommended applying topical cream
放射性灼伤需要数天或数周才会显现
- Radiation burns take days/weeks to present
患者在一段时间后联系了律师
- Patient contacted lawyers some time later
在诉讼结束前，患者在一场车祸中丧生
- Before lawsuit finished, patient died in car accident

这起事故展示了：

1. 对技术系统的过度信任

医生认为：“现代机器不会出错。”

因此误判病人的真实伤情。

Summary of Accidents

两个月后。一名女性因宫颈癌接受治疗，臀部遭受严重放射性灼伤

(2) Two months later. Woman treated for carcinoma of the cervix. Suffered severe radiation burn to hip

- Patient died shortly after 患者不久后去世
尸检表明死因是癌症，而非放射性灼伤
- Autopsy showed cause of death cancer, not radiation
若她能活下来，臀部因灼伤需要置换
- Had she lived, hip would need replacing due to burns

12 月：放射性过量导致严重灼伤

(3) December: Radiation overdose causes severe burn

患者 10 年后仍健在

- Patient still living 10 years later

(4,5,6) Three fatal overdoses 三例因放射性过量致死的病例

所有事故共同揭示的重要特点

1. 前期事故没有被正确调查 导致更多、更严重的事故

第一起事故如果被认真调查，后面 5 起事故可能完全不会发生

2. 医生、操作员、公司对设备过度信任 (overtrust) 医护人员不相信机器会错 AECL (制造商) 拒绝承认软件有问题在事故初期，所有人都“否认可能是机器错”

3. 辐射烧伤延迟显现，使得事故难以识别：导致对受害者的怀疑、对设备的信任增加。

4. 公司未向监管机构报告事故：导致外界无法得知类似事故已发生多次。

5. 软件完全缺乏防护机制（完全依赖软件：没有硬件安全锁 (hardware interlock-->软件 bug = 致命事

6. 并发条件竞争 (race conditions) 导致剂量计算错误:这是后来调查发现的关键软件问题。

问责制与道德责任

Accountability & Moral Responsibility

Douglas Birsch (ethicist and author) analysed the case

以下人员应承担道德责任

- The following people bear **moral** responsibility
 - The programmer who designed software 设计软件的程序员
 - The company's quality assurance officer 公司的质量保证官员
- **Not legal** responsibility 但不承担法律责任
 - That is a matter for the courts to decide 这应由法院来判定

Birsch's Moral responsibility

一个人要承担道德责任，必须满足三个条件

3 criteria **must be met** for person to bear **moral responsibility**

1. “The person’s **actions must have caused the harm, or been a significant causal factor**”
1. 该人的行为必须造成了损害，或者是一个重要的促成因素。
2. The person must **have intended or willed the harm, or it must be a result of his/her negligence, carelessness, or recklessness**
2. 此人必须有意或故意造成伤害，或者这种伤害必须是由于其疏忽、粗心大意或鲁莽所致。
3. The person must be able to have known, or must **know the consequences of the action, or must have deliberately remained ignorant of them**”
3. 这个人必须能够知晓，或者必须知晓该行为的后果，或者必须故意对其后果一无所知。

Qu: Do these criteria make the programmer(s) morally responsible?

Moral responsibility of the programmer

- **Birsch's argument:**

1. 程序员制造了，或者至少是允许了那些直接导致辐射过量的软件错误。

1. The programmer made, or at least allowed, the software errors that directly caused radiation overdose

2. 作为软件专家，他应该知道复杂的软件可能存在测试难以发现的漏洞。他本应将这一事实告知管理层。

2. As software expert, he ought to know that complex software could have bugs unlikely to be detected by testing. He should have notified management of this fact

3. 程序员知道机器可能具有致命性，并且明白软件故障可能带来的后果。

3. Programmer knew machine potentially lethal and understood possible consequences of software failure

Therac-25: Lessons learned

- AECL did not have the **software code independently reviewed**
- AECL did not consider the design of the software during its assessment of how the machine might produce the desired results and what failure modes existed. These form parts of the general techniques known as **reliability modeling and risk management**
- The system noticed that something was wrong and halted the X-ray beam, but merely displayed the word "MALFUNCTION" followed by a number from 1 to 64. **The user manual did not explain or even address the error codes**, so the operator pressed the P key to override the warning and proceed anyway.
- AECL personnel, as well as machine operators, initially did not believe complaints. This was likely due to **overconfidence**.
- AECL had **never tested** the Therac-25 with the combination of software and hardware until it was assembled at the hospital.

Now software engineering methods have matured to hopefully
stop many of these problems

🔥 顶级总结：Therac-25 出错的真正原因是什么？

总结起来：

- ✓ 没有独立代码审查
- ✓ 没有风险建模
- ✓ 软件错误处理不完善
- ✓ 错误信息模糊导致操作员误用
- ✓ 医护人员不相信病人 → 过度自信
- ✓ 没做系统级软硬件测试
- ✓ 软件替代硬件保护，但软件有严重缺陷

换句话说：

这不是一个 bug，而是整个工程流程的全面失败 (system engineering failure)。



University of
Nottingham

UK | CHINA | MALAYSIA

Malware

Malware 恶意软件

蠕虫——自我复制并通过网络传播以感染其他计算机

- **Worm**—makes copies of itself and propagates through network to infect other computers

病毒——与蠕虫类似，但寄生于其他程序。只有当程序运行时病毒才会传播

- **Virus**—similar to worm, but resides in another program. Program must execute for virus to propagate

间谍软件——秘密安装的程序，用于收集计算机用户的信息

- **Spyware**—program secretly installed for purpose of collecting information about computer's user(s)

木马——伪装成无害甚至有用的软件，实则具有恶意

- **Trojan Horse**—software masquerades as innocent and perhaps useful, but actual malicious

根套件——嵌入操作系统并获取特殊权限的程序。有可能凌驾于操作系统的权限之上，获取密码和所有文件的访问权限

- **Rootkit**—program that embeds into OS and acquires special privileges. Potential to override authority of OS, gaining access to passwords and all files

★ 整页内容的核心对比总结

类型	是否自复制	是否依赖宿主程序	是否网络传播	是否伪装	是否高危（控制系统）
Worm	✓	✗	✓ 自动传播	✗	中危
Virus	✓	✓(必须附着)	✗ 需要执行	✗	中危
Spyware	✗	✗	不一定	隐藏	中高危（隐私）
Trojan	✗	✗	不一定	✓(伪装)	高危
Rootkit	✗	✗(在内核运行)	不一定	隐藏	★ 极高危（完全控制系统）

“Ethical” Hacking 道德黑客

“黑客”：编写和部署恶意软件的人

- “Hackers”: people who write and deploy malware
“恶意软件”表明恶意，所以直接不道德
- “Malware” indicates malice, so directly immoral
 - To vandalise other’s property, steal from their credit card, or spy on a person, etc. is clearly immoral 破坏他人财产、盗刷信用卡或监视他人等行为显然是不道德的
- Some “white hat” hackers have legitimate reasons
一些“白帽”黑客有正当理由
在恶意攻击者发现之前寻找系统漏洞
 - To seek vulnerabilities in system before malicious attacker finds them
- 风险：我们如何知道他们没有做不正当的事？或者，如果这个人曾因非法黑客行为被定罪，那又会怎样？
 - Risk: how do we know they are not doing illegitimate things? Or, what happens if person was originally convicted of illegal hacking?
 - Sometimes ethical hacking violates the law – you need legal authority
有时道德黑客行为会违反法律——需要法律授权
 - Education & training of ethical hackers is controversial
道德黑客的教育和培训存在争议
- Zero-day attack 零日攻击
 - Attack that exploits previously unknown vulnerability 利用先前未知漏洞的攻击
- Humans often a weak link in system security 人类往往是系统安全中的薄弱环节
 - Phishing, tricking users, listening devices & cameras 网络钓鱼、欺骗用户、窃听设备和摄像头



University of
Nottingham

UK | CHINA | MALAYSIA

Summary

Summary

我们在日常生活中都信赖计算系统

- We all **trust** computing systems in our daily lives
关乎我们的健康、安全、银行、安保等等
 - With our health, safety, banking, security, ...失败可能影响到我们所有人并造成痛苦/死亡
- **Failure** can affect us all and cause pain / death
 - Safety-critical systems 安全关键型系统失败是可以被最小化的
- Failure can be **minimized**
 - Good software engineering / testing 良好的软件工程/测试
 - Professional and ethical training 专业的和道德的培训
- As **professionals**, we may bear **moral responsibility**
作为专业人士，我们可能要承担道德责任

Additional Reading

- Chapter 5 of the book *Ethics in a Computing Culture* (Brinkman and Sanders, 2013)
- Chapter 8 of the book *A Gift of Fire* (Baase, 2013)
- Chapter 8 of the book *Ethics for the Information Age* (Quinn, 2013)
- Chapter 6 of the book *Ethics and Computing* (Bowyer, 2001)
- An Investigation of the Terac-25 Accidents. N.G. Leveson, G.S. Turner. *IEEE Computer*, Vol. 26, No. 7, pp. 18-41, 1993.
- *Exploratory Engineering in AI*. Luke Muehlhauser and Bill Hibbard. *Communications of the ACM*, Vol. 57, No. 9, pp. 32-34, September 2014.

Workshop

- Case study.
- Investigate causes of software failure and responsibility.

